

**Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій**

**ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ №6
«Автоматизоване тестування Python-програм»
з курсу «Професійний Python»**

Варіант №8 — «Система бронювання готелю»

Виконав: студент 3 курсу
групи ФєП-32
Івасів Данило

Перевірів: доцент
Куньо Іван Михайлович
Дата: 05.10.2026

1. Мета роботи

Сформувані практичні навички професійного автоматизованого тестування Python-програм із використанням фреймворку `pytest`, навчитися проєктувати unit та integration tests, створювати fixtures, використовувати параметризацію, перевіряти exceptions, ізолювати файлову систему та зовнішні залежності, застосовувати mocks і аналізувати test coverage.

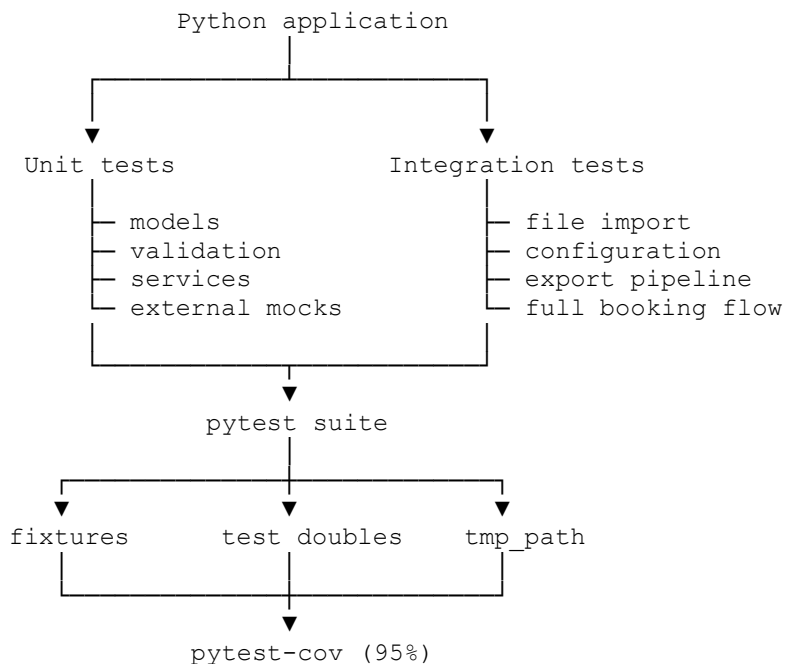
2. Індивідуальний варіант і постановка задачі

Варіант №8 — «Система бронювання готелю».

Завдання:

- Протестувати створення Room/Booking та валідацію price/status.
- Перевірити логіку створення бронювання, duplicate booking, скасування та зміну availability.
- Реалізувати boundary та invalid tests для цін і кількості ночей.
- Протестувати CSV import та JSON export використовуючи `tmp_path`.
- Використати `monkeypatch` для конфігурації allowed statuses.
- Застосувати Mock для Payment/Notifier (перевірка failure path та interaction assertion).
- Реалізувати AsyncMock для асинхронного external booking gateway.
- Отримати coverage report із branch coverage не менше 80%.

3. Підсумкова схема тестування проєкту



4. Структура проєкту

```
hotel_project/
├── pyproject.toml
├── uv.lock
├── src/
│   └── hotel/
│       ├── __init__.py
│       ├── models.py           # Доменні моделі (Кімнати, Бронювання) та винятки
│       ├── config.py          # Конфігурація доступних статусів
│       └── exporters.py       # Робота з файловою системою (CSV, JSON)
```

```

├── services.py      # Бізнес-логіка, репозиторії, зовнішні інтерфейси
└── test/
    ├── conftest.py  # Спільні фікстури (shared fixtures)
    ├── unit/        # Ізольовані модульні тести
    │   ├── test_models.py
    │   ├── test_services.py
    │   └── test_config.py
    └── integration/  # Інтеграційні тести (взаємодія компонентів)
        ├── test_exporters.py
        └── test_booking_flow.py

```

5. Основні модулі та тести

5.1. Fixtures (conftest.py)

```

import pytest
from src.hotel.models import StandardRoom, Suite, Booking

@pytest.fixture
def standard_room():
    return StandardRoom(room_number="101", price=1000.0)

@pytest.fixture
def valid_booking(standard_room):
    return Booking(booking_id=1, room=standard_room, guest_name="Ivan", nights=3)

```

5.2. Boundary testing, Parametrization та pytest.raises (test_models.py)

```

@pytest.mark.parametrize("price", [0, -100, -0.01])
def test_invalid_room_price(price):
    with pytest.raises(ValueError, match="positive"):
        Room(room_number="101", price=price)

def test_suite_total_price_with_tax(suite_room):
    booking = Booking(1, suite_room, "Guest", 2)
    # 3000 * 2 = 6000 + 5% tax = 6300
    assert booking.total_price() == pytest.approx(6300.0)

```

5.3. Mock, side_effect та AsyncMock (test_services.py)

```

def test_payment_failure_path(valid_booking):
    repo = BookingRepository()
    payment = Mock(spec=PaymentGateway)
    payment.charge.return_value = False # Failure path
    notifier = Mock(spec=Notifier)

    service = BookingService(repo, payment, notifier)
    with pytest.raises(BookingError, match="Payment failed"):
        service.create_booking(valid_booking)
    notifier.send.assert_not_called() # Interaction assertion

@pytest.mark.asyncio
async def test_async_gateway_success():
    client = AsyncMock()
    client.get_status.return_value = {"status": "confirmed"}
    gateway = AsyncBookingGateway(client)
    status = await gateway.fetch_external_status(999)
    assert status == "confirmed"
    client.get_status.assert_awaited_once_with(999)

```

5.4. monkeypatch (test_config.py)

```
def test_allowed_statuses_custom(monkeypatch):
    monkeypatch.setenv("ALLOWED_STATUSES", "active, archived")
    assert get_allowed_statuses() == ["active", "archived"]
```

5.5. tmp_path (test_exporters.py)

```
def test_json_export(tmp_path, bookings_list):
    output_file = tmp_path / "bookings.json"
    data = [{"id": b.booking_id, "guest": b.guest_name} for b in bookings_list]

    export_bookings_to_json(data, output_file)

    assert output_file.exists()
    saved_data = json.loads(output_file.read_text(encoding="utf-8"))
    assert saved_data[0]["guest"] == "Anna"
```

6. Результат роботи тестів та аналіз покриття (Coverage)

```
===== test session starts
=====
collected 18 items
test/integration/test_booking_flow.py::test_full_booking_pipeline PASSED
...
test/unit/test_services.py::test_async_gateway_failure PASSED

===== tests coverage
=====
Name                               Stmts   Miss Branch BrPart  Cover   Missing
-----
src\hotel\__init__.py               0       0      0      0    100%
src\hotel\config.py                 4       0      0      0    100%
src\hotel\exporters.py              13       0      2      0    100%
src\hotel\models.py                28       1      6      1     94%    20
src\hotel\services.py               36       2      6      1     93%    24, 35
-----
TOTAL                               81       3     14      2     95%
===== 18 passed, 1 warning in 0.18s
=====
```

7. Експериментальна частина

Експеримент 1. Weak vs Strong tests (execution ≠ verification)

Створено тест, який викликає функцію створення бронювання без жодних assert. Тест проходить успішно, але результат залишається неперевіреном. Після додавання assert (перевірка зміни статусу на confirmed та is_available на False) тест став strong, гарантуючи коректність логіки, а не лише відсутність синтаксичних помилок.

Експеримент 2. Branch coverage vs Statement coverage

При валідації ціни кімнати (if price <= 0: raise ValueError) Statement coverage досягав 100% лише при тестуванні некоректної ціни. Після увімкнення --cov-branch виявлено, що гілка з коректною ціною не має повного покриття. Завдяки цьому було додано happy-path тест для Room.

Експеримент 3. Mock failure path

Замість класичного return_value=True для PaymentGateway, було використано return_value=False (або side_effect). Перевірено, що BookingError піднімається коректно, а

взаємодія з Notifier не відбувається (`assert_not_called`), що запобігає надсиланню хибних повідомлень клієнтам про успішну оплату.

Експеримент 4. Ізоляція файлової системи (`tmp_path`)

Виконано декілька запусків файлових тестів (`test_json_export`). Оскільки `tmp_path` гарантує створення унікальної директорії для кожного тесту, відсутні конфлікти паралельного запису або вплив старих даних (`garbage`) на результати нових тестів.

8. Відповіді на контрольні запитання

1. Що таке `unit test` та `integration test`?

`Unit test` перевіряє невелику одиницю поведінки ізольовано (наприклад, метод класу). `Integration test` перевіряє взаємодію декількох компонентів разом (наприклад, конфігурацію, сервіс і файловий експорт).

2. Для чого `pytest.mark.parametrize`?

Для запуску одного і того ж тесту з різними наборами вхідних даних. Це усуває дублювання коду і дозволяє легко перевірити цілий ряд `boundary cases`.

3. Для чого `tmp_path`?

Це вбудована фікстура `pytest`, яка створює ізольовану тимчасову директорію для кожного тесту, усуваючи конфлікти файлів та запобігаючи забрудненню реальної файлової системи.

4. Що таке `monkeypatch`?

Фікстура, яка дозволяє тимчасово підміняти змінні середовища (`environment variables`), атрибути класів чи функції під час виконання тесту. Після завершення тесту зміни автоматично скасовуються.

5. Що таке `Mock` і для чого `return_value` / `side_effect`?

`Mock` — це об'єкт, який імітує поведінку реальної залежності (наприклад, платіжного шлюзу). `return_value` задає фіксоване значення, яке поверне мок, а `side_effect` дозволяє підняти виняток (імітація збою) або повернути різні значення при послідовних викликах.

6. Як перевірити, що метод викликано один раз?

За допомогою `interaction assertions`: `mock_object.method.assert_called_once()` або `assert_called_once_with(args)`.

7. Що таке `AsyncMock`?

Це спеціальний клас з `unittest.mock`, призначений для імітації асинхронних функцій та залежностей. Перевірка виклику здійснюється через `assert_awaited_once_with()`.

8. Для чого `pytest.raises`?

Для перевірки `exception path`: він переконується, що при передачі невалідних даних (наприклад, від'ємної ціни) код гарантовано підніме очікуваний тип винятку.

9. Що таке `branch coverage`?

На відміну від `statement coverage` (чи виконався рядок), `branch coverage` показує, чи були перевірені всі можливі відгалуження умовних операторів (наприклад, `i true`, `i false` для `if`).

10. Чи гарантує 100% coverage відсутність bugs?

Ні. Coverage лише показує, який код виконувався під час тестів. Без якісних перевірок результатів (assert) код може виконуватися на 100%, але приховувати логічні помилки.

9. Висновок

У лабораторній роботі №6 було здійснено перехід від ручної перевірки Python-програми до систематичного автоматизованого контролю її поведінки за допомогою фреймворку pytest. Розроблено повноцінний test suite для системи бронювання готелю (18 тестів, 95% покриття).

Використання фікстур дозволило ефективно перевикористовувати тестові дані, а параметризація — лаконічно покрити граничні значення. Завдяки tmp_path та monkeypatch було безпечно протестовано файлові операції та конфігурацію без впливу на реальне середовище. Ізоляція зовнішніх залежностей (оплата, сповіщення) через Mock та AsyncMock дозволила надійно перевірити як успішні сценарії, так і шляхи відмови (failure paths). Аналіз покриття за допомогою pytest-cov підтвердив високу якість тестів, що забезпечує надійний захист проєкту від регресій при подальшому рефакторингу.